

3-Party Asynchronous Communication with Perfect Secrecy based on balancing the middle party

Introduction

Team Members

- Adnan Jakati: mj3184@nyu.edu
- Dhanraj Chavan: dsc7792@nyu.edu
- Mayank Ramnani: mr7172@nyu.edu

Task Distribution

- Mayank Ramnani: Protocol Design, Implementation, Proof Intuition
- Adnan: Implementation, Testing
- Dhanraj: Documentation, Report Writing

Chosen Value of m :

$m = 3$

Claims

1. Our protocol ensures perfect secrecy.
2. Theoretical maximum pad wastage in worst-case scenarios is limited by a function of d , particularly improving on the baseline split approach of $2n/3$.
3. Wastage in the worst case scenario (d undelivered messages) is $\min(n, 3*d)$

Scenarios:

1. Scenario (a): Only one, randomly chosen, party repeatedly sends L -length messages.
Pads wasted: $3*d$
2. Scenario (b): Only two, randomly chosen, parties repeatedly send L -length messages, where the decision of who sends the next message is also randomly chosen.
Pads wasted: $2.5d$ (average case)
3. Scenario (c): Each message is sent by a randomly chosen party out of the 3 parties.
Pads wasted: $2.7d$ (averaged across 10000 runs)
 - Maximum wastage across all possible usage schedules: **$3d$**

Informal Explanation

- N is the number of pads available. These are shared between all the parties, with each party having the same list at the start of the protocol.
- For $m=3$, there will be 3 parties. If the available pads are treated as a list, the first party (A) will use pads from the left, moving right; second party (B) will use the pads in the middle, third party (C) will use the pads starting from the right end of the list, moving left.
- As pads are used, they are removed from the available list of the sender.
- When messages are sent, the index of the pad used is sent along with them so that decryption at the receiver is possible.
- The receiver also updates its available list of pads and removes the pads whose index it receives since they have been used elsewhere.
- This protocol needs to stop a client from sending a message when
$$\text{available_pads_at_client} \leq 3d$$
to avoid breaking perfect secrecy.
- Thus, max wastage is of $3d$ pads.

Rigorous Description

Pseudocode:

```
initialize N pads as a shared list among all parties
set m = 3 // number of parties
set d // security parameter for stopping condition
```

```
define party A:
    uses pads from left to right
```

```
define party B:
    uses pads from the middle
```

```
define party C:
    uses pads from right to left
```

each party maintains its own available_pads list, initially equal to the shared list

```
function send_message(sender, receiver, message):
    if length(sender.available_pads) ≤ 3d:
        abort transmission // stop sending to maintain secrecy
```

```

pad_index = select_pad(sender)
pad = sender.available_pads[pad_index]

encrypted_message = encrypt(message, pad)

send (encrypted_message, pad_index) to receiver

remove pad_index from sender.available_pads

function receive_message(receiver, encrypted_message, pad_index):
    if pad_index not in receiver.available_pads:
        abort // integrity failure

    pad = receiver.available_pads[pad_index]
    message = decrypt(encrypted_message, pad)

    remove pad_index from receiver.available_pads

    return message

function select_pad(party):
    if party is A:
        return leftmost available index
    if party is B:
        return middle available index
    if party is C:
        return rightmost available index

```

Explanation

This pseudocode describes a secure communication protocol where three parties (A, B, and C) share a list of N one-time pads. Each party selects pads uniquely to avoid conflicts:

- **A** takes pads from the left.
- **B** takes from the middle.
- **C** takes from the right.

When a party sends a message, it selects a pad, encrypts the message, sends the ciphertext along with the pad index, and removes the pad from its list. The receiver decrypts using the corresponding pad and then also removes it.

To maintain perfect secrecy, a party stops sending when its available pads drop to $\leq 3d$. This ensures a maximum wastage of $3d$ pads while preventing reuse that could compromise security.

Proof Intuition

One-Time Pad Security:

Each pad is a random key that, when used only once, guarantees perfect secrecy. The encryption uses a fresh pad every time, so the ciphertext leaks no information about the plaintext.

Dynamic and Non-Overlapping Pad Selection:

- **Party A** always selects the leftmost available pad.
- **Party C** always selects the rightmost available pad.
- **Party B** dynamically chooses the middle available pad.
 - Unlike static partitions where each party has a fixed subset of pads, here the available list is shared and changes as pads are removed.
 - There always needs to be a gap of (d) between A, B and B, C to prevent collisions.
 - Additionally, since B may have (d) undelivered messages, an additional gap of $(d/2)$ on both sides is required leading to a total gap of $(d+d/2)$ on both sides of B.
- Thus, $(2 * (1.5d)) = (3d)$ is the total amount of wastage of pads.

Synchronized Pad Removal:

After a pad is used for encryption, the sender removes it from their available list. Once a message is received that is encrypted with a given pad, the receiver removes it from its available list. This synchronized removal is critical for ensuring that each pad is used only once, maintaining the one-time pad's requirement for perfect secrecy.

Perfect Secrecy Assurance:

The protocol enforces a safety cutoff: when a party's available pads drop to or below $3d$, message transmission for it stops. This threshold prevents the system from reaching a state where pad reuse might occur, which would compromise secrecy.

By ensuring that each message is encrypted with a fresh, randomly selected pad—and by coordinating pad selection and removal across all parties—the protocol preserves the ideal properties of the one-time pad. Even if an adversary intercepts a message, the randomness and one-time use of the pads prevent any leakage of plaintext information.

README

Initializing the repository

1. Install poetry if you haven't already: `brew install poetry` OR `apt install poetry`
2. Install all required dependencies of project: `poetry install`

```
root@ubuntu-cloud:~/secure-3-party-comms# poetry install
Installing dependencies from lock file

Package operations: 10 installs, 0 updates, 0 removals

  - Installing six (1.17.0)
  - Installing iniconfig (2.0.0)
  - Installing numpy (2.2.3)
  - Installing packaging (24.2)
  - Installing pluggy (1.5.0)
  - Installing python-dateutil (2.9.0.post0)
  - Installing pytz (2025.1)
  - Installing tzdata (2025.1)
  - Installing pandas (2.2.3)
  - Installing pytest (8.3.4)

Installing the current project: secure-3-party-comms (0.1.0)
root@ubuntu-cloud:~/secure-3-party-comms# vim README.md
root@ubuntu-cloud:~/secure-3-party-comms# poetry run pytest
===== test session starts =====
platform linux -- Python 3.12.3, pytest-8.3.4, pluggy-1.5.0
rootdir: /root/secure-3-party-comms
configfile: pyproject.toml
collected 5 items

tests/test_scenario_a.py ...
tests/test_scenario_b.py .
tests/test_scenario_c.py .

===== 5 passed in 5.01s =====
```

Running the protocol

- The protocol implementation is a module that can be imported using: `from secure_3_party_comms.protocol import *`
- To change the values of N, d or the probability of a message being undelivered, change the global variables at the top of `secure_3_party_comms/protocol.py`

Running the testcases

- To run all testcases, run `poetry run pytest`
- To run testcases without output suppression: `poetry run pytest -s`
- To run a particular scenario: `poetry run pytest tests/test_scenario_a.py`

- To run a custom scenario, follow the instructions:
 - i. Create a new file in the `tests` directory
 - ii. Add `from tests.helpers import *` at the top of the file to import protocol functionality
 - iii. Define a function with name starting with `test_` which calls the helper function `run_test_case` with appropriate parameters. Example from `test_scenario_b.py`: `run_test_case(sending_clients=['A', 'B'], prob_distribution={'A': 0.5, 'B': 0.5})`

End
